Viver: Visual Verification for Trusting Autonomous UI Rewrites

Youri Reijne
CodeGrade
Amsterdam, The Netherlands
youri.reijne@codegrade.com

Abstract

Modern coding agents handle multi-step tasks well, yet we still drive them one step at a time and check every result ourselves. We ask what it would take to trust an agent to do a behavior-preserving 1:1 dependency rewrite of User Interface (UI) code on its own. The obstacle is verification. A migration can pass every unit and component test while the rendered interface is visibly broken. That is what happened in a real migration regression (PROD-3516): no test caught it, and only a human reviewer's eye did. We build a visual gate from three composable Claude Code skills: 1, light/dark screenshot capture, 2, a reference-free critic, and 3, a before/after critic. An autonomous rewriter wires them into the migration. We evaluate each critic with a blind-subagent harness. Reference-free, the critic detects broken UI at 93% recall and 100% precision, but it can misjudge intent. Given a before/after baseline, it detects 100% over 38 pairs. On a faithful reconstruction of PROD-3516 it flagged the same broken state the reviewer had caught by eye. This verification layer is what moves autonomous rewriting from a reactive Level-2 workflow toward a trustworthy Level-3 one. The human remains at the design gate.

Keywords

LLM agents, autonomous code rewriting, visual regression, verification, Claude Code, e2e testing

1 Introduction

Coding agents now handle multi-step tasks well, yet we still drive them one step at a time and check every result by hand. Letting an agent finish a task unattended runs into a wall of trust. Consider a concrete failure. CodeGrade was migrating its popovers from a third-party library to an in-house top-layer component. The goal was to fix an earlier, separate bug where popovers painted behind the publish modal. One migration was done with Claude's help, and the anchor placement came out wrong. In the running app, the tooltip is detached and rendered in the empty top-left corner. Every component, and e2e test passed. The defect existed only in the integrated UI, and a human reviewer had to catch it by eye in code review (PROD-3516, PR #3757). That is the gap this paper addresses: a Claude-driven rewrite whose broken visual outcome only a human eye caught.

We study a narrow but common task. A behavior-preserving 1:1 dependency rewrite swaps an old dependency for an in-house equivalent without changing behavior. We enforce a strict rule. Test assertions are frozen, and only selectors and mount wiring may change. A changed assertion is by definition a behavioral change. The rewrite itself is mechanical. The hard part is trusting the result without a human watching the screen, because that human is sometimes the only thing that catches the regression.

On the conference's AI-usage spectrum this workflow sits at Level 2: integrated but reactive. We hand the agent a rewrite ticket. The agent then works on its own. It reads the ticket, ensures test coverage, performs the 1:1 rewrite, confirms the tests still pass, visually verifies the result, and opens a pull request with before and after screenshots. It counts as Level 2 only because a human still picks and hands over each ticket. We move towards Level 3, where we could design an automated pipeline for these tickets. The agent spots the tech-debt ticket itself and brings the finished pull request for approval. The pipeline already runs end to end. What makes it trustworthy is the visual verification step. That step is the by-eye review a human reviewer would otherwise do, now run by the agent on itself, and it is this paper's focus.

We frame the work as four research questions, each a reusable building block.

- **RQ1 (capture).** Can we render changed UI to an image automatically, in light and dark?
- **RQ2 (reference-free).** Can a critic judge a single screenshot, with no before image?
- **RQ3 (differential).** Given a before/after pair, can a critic tell a regression from an intended change?
- **RQ4 (autonomy).** Can an agent compose RQ1–RQ3 into a trustworthy 1:1 rewrite?

Two case studies test the questions on real data: a batch of real pull requests for RQ1, and a faithful reconstruction of PROD-3516 for RQ4. Our headline findings are that reference-free detection is strong but can misjudge intent, that a before/after baseline closes that gap, and that on PROD-3516 the gate reproduced the catch the human reviewer made.

2 Approach: Modular Visual Gates

Modular gates. We split the problem into four building blocks, packaged as five Claude Code skills (capture needs two: one for component tests, one for e2e). The rewriter is the skill that composes the others. M1 (capture) renders a target and screenshots it in light and dark. It can run in component isolation or, preferably, integrated in the running app via Cypress. Capture is deterministic. The command `cy.captureThemes` toggles the theme and awaits web fonts. Cypress's fixed default viewport and screenshot-time animation freezing handle the rest, with no new dependencies. The agent then reads the PNGs directly. M2 (reference-free critic) shows the agent one screenshot and asks whether anything is visually wrong. M3 (differential critic) shows a before/after pair and asks whether the change is a regression, an intended redesign, or noise. M4 (autonomous rewriter) performs the 1:1 migration and uses M1–M3 as its self-check before opening a pull request.

Blind evaluation harness. We score the critics blind. Each screenshot goes to one fresh reviewer instance with a neutral prompt and a neutral filename, since a suggestive name primes the

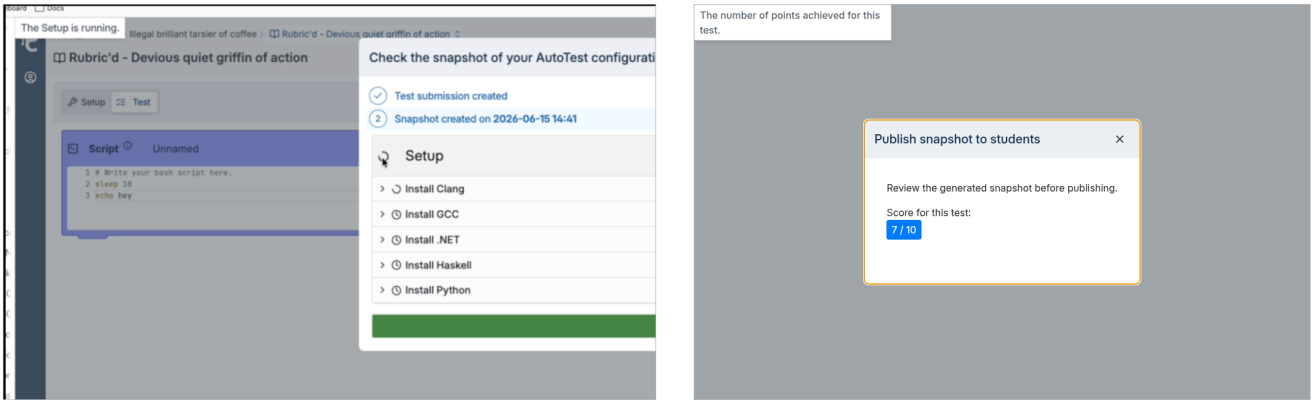


Figure 1: PROD-3516, real defect and reproduction. Left: the real test deployment, where the migrated tooltip ("The Setup is running.") detached to the page's top-left corner while the AutoTest snapshot modal was open. Right: our faithful reproduction with the real components. The blind reference-free gate flagged the reproduction on both runs at 0.88. The defect appears only in the integrated UI. Every test passed, and a human reviewer caught the original. [upload `prod3516-real-vs-repro.png`]

verdict. Each verdict is structured output, scored against ground truth. Faults come from a nine-type catalog: stacking, clipping, off-screen, overlap, missing, low-contrast, wrong-semantic-color, misalignment, and mis-sizing. We split them into gross and subtle tiers and inject them at render time as reversible CSS, so the clean control is genuinely clean.

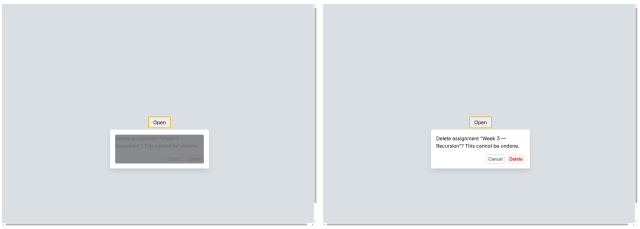


Figure 2: The matched intent test. The same two frames appear in opposite order. The order behind→clean is judged an intended fix, and clean→behind a stacking regression.

3 Results

Table 1 summarizes the evaluation. We describe each result and its one telling failure.

Reference-free detection (RQ2, M2). We tested 42 images: 30 faulted and 12 clean controls, 20 synthetic and 22 from the PROD-3516 reconstruction. The critic reached 93% recall and 100% precision, with zero false alarms on the controls. It caught every gross fault (14/14) and most subtle ones (14/16). It was more confident when right than wrong (0.87 versus 0.74), with no light/dark bias. It even flagged a real, unintended low-contrast defect on a screen we had labeled clean. Its two misses share one cause: a 48 px shift of a whole panel, which it judged intentional. Its free-text `type` label also names the symptom rather than the mechanism. The practical reading is to trust the binary signal as a PR gate and to treat the label as a hint. Pair it with a reference or DOM check where whole-region alignment matters. The critic was primed with the catalog and voted once per image.

Differential judgment (RQ3, M3). Given the before as an appearance specification, the critic classified 38 pairs at 38/38 three-way accuracy. The set held 26 regressions, 6 intended changes, and 6 noise pairs. The critic caught every regression, raised zero false alarms on clean or intended changes, accepted all six redesigns, and named the exact defect type on all 26 regressions. The decisive test is matched. The same two synthetic stacking frames in opposite order are judged oppositely. The order behind→clean reads as an intended fix, and clean→behind reads as a stacking regression, in both themes. The harder real differential is the M4.1 case below, where a correct overlay resembles obstruction. The before/after baseline is what M2 lacked. It catches the 48 px shift M2 excused, and it types defects exactly.

End-to-end autonomy (RQ4, M4). We ran the full skill on a real migration: `LateSubmissionIcon` from a `b-popover` to the in-house `Popover` (PR #3814). Behavior was preserved by construction. The test diff was two selector changes and zero assertion-body changes, and all seven frozen tests passed before and after. The run used two designed checkpoints, a scope gate and the final PR, with no unplanned human intervention. It took about 35 minutes, mostly cold install and stack build; the rewrite itself took minutes. It used on the order of a million tokens. Its self-check was the blind reference-free critic, the implementation that `visual-critic` now packages. We also put two injected negative controls to the gate. It caught a gross clipping regression at 0.92 but rated a subtle low-contrast one only 0.50, calling it a possibly deliberate muted style. That is a true positive at coin-flip confidence, and it would slip under any threshold. The migration itself produced no regression, since before and after were identical. The skill also knows where to stop. A true 1:1 rewrite exists only for hover tooltips with in-repo precedent, so for click, programmatic, or slot usages it aborts rather than guess.

The marquee case (M4.1). We reconstructed the bug with the real components and put it to the blind reference-free gate. The PROD-3516 regression was the tooltip detaching to an empty corner.

Table 1: Evaluation of the visual gate across modules. Bold marks the headline metric.

Module	Eval set	Headline result
M2 reference-free	42 images (30 fault / 12 clean; 14 gross / 16 subtle; 20 synthetic + 22 real PROD-3516)	**93% recall (28/30), 100% precision (28/28), 0/12 false alarms**; gross 14/14, subtle 14/16; conf. 0.87 vs 0.74
M3 before/after	38 pairs (26 regression / 6 intended / 6 noise)	**38/38 (100%) three-way; regression 26/26; 0/6 false alarms on clean and on intended; 6/6 redesigns accepted; exact type 26/26**; conf. 0.93
M4 autonomy	one real migration (PR #3814, `LateSubmissionIcon`)	**selector-only diff (2 selectors, 0 assertion bodies); 7/7 tests green; 0 unplanned interventions**; ∼35 min / ∼1M tokens
M4.1 marquee	PROD-3516 reconstructed	reference-free **caught 2/2 at 0.88** in full-app context (component-only misses); differential gate inverted
M1.1 reach	5 real PRs → drafts #3809–3813	**44 light+dark PNGs**; full coverage on 3 via existing flows; honest gaps logged

In the full-app context, with the modal open, the gate caught it on both runs at 0.88. In component isolation the gate is blind to it, because the orphan sits beside its also-displaced trigger and looks normal. So the gate reproduced the reviewer's catch on exactly the bug a human had to flag. An e2e visual check could stand in for the manual one. A component-only check could not. The separate behind-the-modal occlusion is harder. That was the earlier ticket that motivated the migration. When the popover peeks past the modal the gate catches it (0.82), but fully hidden it sees a clean modal and misses it (0.90). That class needs a functional assertion. Two honest limits remain. The correct fix drew a 0.72 false positive, and the naive before/after differential inverted here, reading the correct overlay as clutter. This is a mechanism-faithful reconstruction, not a replay of the exact publish flow.

Reach (M1.1). We ran the capture skill across five real merged PRs. It produced 44 light and dark screenshots in five draft PRs (#3809–#3813), which we opened to test whether the skill's description-driven coverage reached every changed surface. Three were fully covered by riding existing e2e flows. A fourth was covered only through a flagged component-isolation fallback, because no execution host was available for its in-app render. On the fifth, a username-hiding PR, we logged nine out-of-scope or unreachable surfaces honestly rather than fake them, and unfortunately the setup for the feature was too difficult to automatically add screenshots for in one go. The exercise surfaced and fixed four weaknesses in the skill.

4 Reusability

Every part is built to be reused, not just described. The four building blocks ship as five committed Claude Code skills: `inspect-component-ui` and `pr-screenshots` for capture, `visual-critic` for the reference-free judgement, `visual-diff-critic` for the before/after judgement, and `rewrite-dependency` for the rewriter that composes them. The rewriter skill lands with PR #3814; the rest are on the branch today. Capture is a single Cypress command, `cy.captureThemes`, that adds no dependencies. The evaluation rig is the nine-type fault catalogue in the repo, plus a blind-subagent harness and a 42-image labelled set that are reconstructible from the method. The adoption path for any team with a Cypress suite is concrete. Ride your existing e2e flows to capture the changed UI, then gate it with the reference-free critic. The steps and prompts live in the skills.

5 Discussion

The finding. One result holds across modules. A reference-free critic is a strong detector but a weak judge of intent, and the before/after baseline supplies that intent. M2 detects broken UI at 93% and 100% yet excuses a 48 px shift. M3, given the before, reaches 100% and tells the matched behind and clean frames apart. The before is the appearance specification the single-screenshot critic never had.

Where the human stays. The gate is a safety net, not a replacement for review. The fully-hidden behind-modal occlusion needs a functional assertion the visual gate cannot provide. The M4 control at 0.50 would slip under a threshold. The naive differential inverted on the real PROD-3516 framing. The human keeps the design gate and decides what the UI should look like. The agent gains a regression gate good enough to reproduce the catches a reviewer makes by eye.

Cost. A migration takes about 35 minutes of wall-clock time, mostly cold install and stack build; the rewrite itself takes minutes. It uses on the order of a million tokens. Capture is about a second and adds no dependencies, and the evaluation is a handful of subagent calls. The value is not that the gate catches what people miss, since a human reviewer did catch PROD-3516. The value is that it removes the manual visual pass on every rewrite, which is what makes unattended migration viable at scale.

Honesty. M3's perfect score is on a balanced but non-adversarial set. We did not probe near-threshold contrast, one-to-two-pixel shifts, or intended changes designed to look like defects. The reference-free critic was primed with the catalogue, voted once per image, and saw only a few examples per type. M4's gross and subtle results came from injected controls, not from regressions the migration produced. M4 migrated a different component than PROD-3516's own, and the marquee result is a reconstruction. We report these because the dead ends are part of the result.

6 Future Work

The next step is an adversarial differential set that reconciles M4.1's inversion with M3's perfect score. It would add near-threshold contrast, sub-pixel shifts, and intended changes designed to look like defects, including the overlay that resembles obstruction. Multi-vote critics should lift subtle recall above the 0.50 the low-contrast control drew. Pairing a functional assertion with the visual one would close the fully-hidden case. The destination is a fully autonomous loop that spots the work, rewrites, gates, and opens a PR. For a codebase carrying years of deprecated UI dependencies, an agent that retires them and verifies its own work, with the human only at the design gate, is the Level-3 step the visual gate was built to make safe.

7 Conclusion

We set out to trust an agent with an unattended UI rewrite, and found that the missing ingredient was sight. A visual gate built from a few small skills detects broken UI reference-free, judges intent given a before/after baseline, and reproduces the visual catch a human reviewer made on a real migration regression. It moves the workflow a concrete step toward trustworthy autonomy, and it leaves the human firmly at the design gate.

References

[1] Association for Computing Machinery. *ACM Master Article Template (acmart)*. https://www.acm.org/publications/proceedings-template.
[2] Cypress.io. *Cypress Documentation: Screenshots and Visual Testing*. https://docs.cypress.io.
[3] Anthropic. *Claude Code: Skills and Subagents*. https://docs.claude.com/claude-code.
[4] CodeGrade. *PROD-3516: migrated popover detaches to the page corner* (caught in code review, PR #3757).